

Water Jug Problem

1. Aim

To implement the **Water Jug problem** using a state-space search technique to measure a required quantity of water by using two jugs with different capacities.

2. Problem Statement

The Water Jug problem involves two jugs of different capacities with no measurement markings. The objective is to obtain a specific amount of water by performing operations such as **filling a jug, emptying a jug, and transferring water from one jug to another** without wasting any water.

3. Solution

The problem can be solved using **Breadth-First Search (BFS)** by representing the amount of water in the two jugs as a state. From each state, possible operations such as filling, emptying, and pouring water between the jugs are generated. BFS explores these states systematically until the required amount of water is obtained.

4. Algorithm

1. Start with both jugs empty.
2. Represent the current state as **(Jug A, Jug B)**.
3. Generate all possible operations:

- Fill Jug A.
 - Fill Jug B.
 - Empty Jug A.
 - Empty Jug B.
 - Pour water from Jug A to Jug B.
 - Pour water from Jug B to Jug A.
4. Check whether the new state has already been visited.
 5. If it is a new state, add it to the queue.
 6. Continue exploring the states using BFS.
 7. Stop when the required amount of water is obtained in either jug.
 8. Display the sequence of operations used to reach the solution.

5. Program

```
from collections import deque
```

```
def water_jug(cap_a, cap_b, target):
```

```
    start = (0, 0)
```

```
    queue = deque([(start, [])])
```

```
    visited = {start}
```

while queue:

 (a, b), path = queue.popleft()

 if a == target or b == target:

 return path + [(a, b)]

states = [

 ((cap_a, b), "Fill Jug A"),

 ((a, cap_b), "Fill Jug B"),

 ((0, b), "Empty Jug A"),

 ((a, 0), "Empty Jug B")

]

Pour A to B

amount = min(a, cap_b - b)

states.append(

 ((a - amount, b + amount), "Pour A to B")

)

Pour B to A

amount = min(b, cap_a - a)

```
states.append(  
    ((a + amount, b - amount), "Pour B to A")  
)
```

```
for state, action in states:  
    if state not in visited:  
        visited.add(state)  
        queue.append(  
            (state, path + [(a, b, action)])  
        )
```

```
return None
```

```
# Jug capacities: 4 litres and 3 litres
```

```
# Target: 2 litres
```

```
solution = water_jug(4, 3, 2)
```

```
print("Solution Path:")
```

```
for state in solution:
```

```
if len(state) == 3:  
    a, b, action = state  
    print(f'{action}: Jug A = {a}, Jug B = {b}')  
else:  
    a, b = state  
    print(f'Goal State: Jug A = {a}, Jug B = {b}')
```

6. Result

The Water Jug problem was successfully implemented using **Breadth-First Search**. The program generates different possible states by filling, emptying, and transferring water between the two jugs. It successfully finds a sequence of operations to obtain the required **2 litres of water** using a **4-litre jug** and a **3-litre jug**.

Result:

The required amount of **2 litres** of water is successfully measured.

